

Evidence-Bound Scoring and Verification in a Scenario-Driven Cyber Range

Navinkumar M, Girinath K, Natarajan V, Heamanthraj S, Sabarinathan V, Nabin Sil

School of Innovation, KGISL Institute of Technology, Coimbatore, India

Abstract

A cyber-range score is meaningful only when the underlying telemetry supporting that score is authoritative, correctly scoped, strictly typed, traceable to persisted events, and reproducible. In scenario-driven training environments, particularly those simulating operational technology (OT) and industrial control systems (ICS), a visually complete dashboard can easily obscure fundamental gaps in telemetry provenance and admission integrity. We investigate a frozen cyber-range prototype (213 source files matching commit 8f426d631f423176063bf2ffac9d2af904f308c7) designed around an OpenPLC/Modbus reconnaissance scenario (PLC-001). Testing the received modules against sixteen declared control requirements revealed eleven critical failures: students could inject unauthenticated intrusion detection system (IDS) events, submit telemetry into unowned or closed exercises, bypass analysis length checks via malformed arrays or untrimmed whitespace, have configured zero score thresholds overridden by JavaScript logical-OR defaults, and obtain clean-state reset certification without verified residue eradication. To resolve these vulnerabilities, we developed a candidate implementation introducing collector-only shared-secret authority, exercise ownership and active-status guards, strict scenario binding, typed string payload bounds, explicit nullish threshold evaluation, and reset abstention when verification evidence is absent. Evaluated against deterministic SQLite-backed test fixtures and loopback HTTP checks, the received prototype satisfied only 5 of 16 requirements, whereas the candidate implementation satisfied 16 of 16. Across 64 scripted exercise fixtures, the candidate verified 256 deterministic score stages (25, 50, 75, and 75/100), 224 unique event-to-objective links, and 64 unaltered raw-event retention checks. In addition, 17 of 17 loopback HTTP security checks passed, and disk database persistence was verified after normal closure and reopening. Crucially, this study is strictly bounded to software- and control-plane verification. The evaluation involved zero human participants, zero live Docker container executions, zero AI-provider calls, and no real physical PLC or Modbus network attacks; submitted containment labels and 50-character text strings verify submission format rather than incident-response quality or physical containment. Our contribution is an evidence-oriented engineering method for identifying, demonstrating, and correcting admission and scoring control weaknesses in a cyber-range prototype.

Keywords: *Cyber range, telemetry authority, evidence provenance, scoring controls, clean-state reset, regression testing, operational technology security.*

1. Introduction

Cyber ranges serve as vital training environments for developing and validating defensive cybersecurity skills across enterprise and critical infrastructure domains [1]. By simulating realistic network topologies, attack vectors, and operational systems, ranges enable practitioners to identify threats, execute incident response playbooks, and practice mitigation strategies without placing production infrastructure at risk. However, the integrity and credibility of any educational or competitive cyber range hinge fundamentally upon its assessment model: a participant's score must reflect genuine, verified defensive competence supported by uncompromised, authenticated evidence.

A persistent engineering challenge in cyber-range design is the divergence between front-end presentation and underlying telemetry provenance. A dashboard may display elaborate scorecards, progress bars, and operational timelines that appear completely convincing to instructors and trainees. Yet, if the telemetry ingestion pipeline fails to validate who generated an event, whether the event belongs to an active exercise, or whether submitted data conforms to rigorous type specifications, the resulting score becomes an artifact of software illusion rather than defensive achievement. This concern is exceptionally pronounced in Operational Technology (OT) and Cyber-Physical System (CPS) environments [3]. In industrial automation networks governed by protocols such as Modbus/TCP, improper assumptions regarding process state or telemetry authenticity carry severe real-world safety, environmental, and physical consequences. Misplaced confidence fostered by flawed range telemetry can lead trainees to overestimate their defensive capabilities when confronting real industrial threats.

This paper investigates this gap through a rigorous, evidence-bounded study of a scenario-driven cyber-range prototype known as CyberPro (Project P282 / P-2024-28-CS-118). Rather than advancing broad, unverified claims regarding enterprise scalability or pedagogical efficacy, we present a focused engineering case comparing the received implementation of the platform against a corrected candidate implementation. We structure our investigation around four bounded research questions:

- RQ1 (Received Control Enforcement): Do the received telemetry admission, scoring, and reset modules enforce their declared security and evaluation rules when tested against targeted counterexamples?
- RQ2 (Candidate Control Remediation): Do the corrected evidence-admission, ownership, typing, and reset-abstention controls prevent the identified counterexamples from contaminating score calculations?
- RQ3 (Score Reconstructability and Evidence Retention): Can final score outputs and objective linkages be deterministically reconstructed from persisted SQLite events without modifying raw underlying audit records?
- RQ4 (Evidence Boundary Characterization): What technical conclusions can be legitimately supported by bounded software-control-plane evaluation, and what operational claims must be explicitly disclaimed?

Importantly, this work does not claim to establish human learning outcomes, educational effectiveness, or real-world industrial containment. The contribution is strictly an empirical, software-level demonstration of how evidence admission, scoring derivation, and environment reset certification can be engineered with scientific defensibility and audit traceability.

2. Project Context and Related Work

The CyberPro project originated as a developmental cyber-range initiative within the School of Innovation at KGISL Institute of Technology, selecting a containerized virtual range architecture over physical hardware testbeds and simulated AI smart-grid environments. The core scenario, designated PLC-001, was formulated to provide hands-on practice detecting Modbus/TCP reconnaissance against an OpenPLC runtime controlling simulated oil-well pumping infrastructure (the OilSprings lab). Our investigation commenced with an exhaustive audit of all project assets, spanning 213 source files corresponding exactly to public Git commit 8f426d631f423176063bf2ffac9d2af904f308c7, six developmental milestone dossiers, ten tracked engineering actions, and associated feedback sheets.

A critical finding from our archival review was a substantial discrepancy between historical project claims and available empirical evidence. Early milestone reports and presentation decks asserted that the platform had been validated across six or seven student users, achieved a 90% task completion rate, and reduced threat detection latency from approximately six minutes to four minutes. However, physical inspection of the linked feedback repository revealed exactly two user survey responses, each recording an opinion score of 4 out of 5, with no underlying timestamped event logs, completion denominators, or trial protocols. Consequently, this revised paper explicitly removes those historical figures from its experimental findings, treating them strictly as unverified historical background that motivates rigorous, automated evaluation.

Our work builds upon several foundational bodies of literature. Yamin et al. established comprehensive taxonomies for cyber ranges, categorizing platforms across scenario design, functional tools, user roles, and evaluation systems [1]. They highlighted that automated assessment requires deterministic telemetry capture. Chindrus and Caruntu documented a red-and-blue team cybersecurity competition platform [2], emphasizing that competition scoring requires objective data pipelines; however, their reported human outcomes cannot be transferred to an unvalidated student prototype. The National Institute of Standards and Technology (NIST) Special Publication 800-82 Rev. 3 provides definitive guidance on OT security [3], establishing that industrial systems require defense-in-depth, strict protocol boundaries, and absolute telemetry integrity due to physical process hazards. The NICE Workforce Framework (NIST SP 800-181 Rev. 1) categorizes cybersecurity competencies into discrete Tasks, Knowledge, and Skills (TKS) [4], underscoring that submitting a text label does not equate to demonstrating operational competence. Finally, NIST SP 800-218 (Secure Software Development Framework, SSDF v1.1) provides the methodological foundation for our code audit, counterexample retention, and regression verification [5]. We do not claim global architectural novelty; rather, we contribute a transparent, reproducible case study in securing cyber-range scoring pipelines.

3. Problem Definition and Evidence-Admission Threats

In a scenario-driven cyber range, an assessment engine converts raw system observations into student scores through an automated pipeline: Sensor Telemetry → Ingestion Route → Event Persistence → Objective Evaluation → Score Aggregation. If this pipeline assumes implicit trust in submitted payloads, it becomes susceptible to severe evidence-admission threats:

1. Telemetry Source Spoofing: Trainees can bypass detection tasks by forging events attributed to automated sensors (such as network intrusion detection systems or log collectors), effectively scoring points without triggering or analyzing actual network traffic.
2. Cross-Exercise Contamination: When ingestion routes omit strict session and exercise identifier checks, events submitted by one trainee can be bound to another trainee's exercise, corrupting scores across parallel sessions.
3. Out-of-Scope and Lifecycle Invalidation: Telemetry submitted to completed, aborted, or uninitialized exercises may still trigger objective evaluation if status guards are not enforced at insertion time.
4. Scenario Mismatch: Submitting telemetry intended for a different scenario definition can allow incompatible objectives to pass if scenario identifiers are unvalidated.
5. Payload Type Coercion and Length Bypasses: Dynamic language type coercion (such as JavaScript's handling of arrays, objects, and strings) permits malformed payloads—such as empty objects or arrays with length properties—to satisfy naive text length assertions without providing real investigative narrative.
6. Threshold Override via Falsy Coercion: Using logical-OR operators (e.g., `threshold || default`) inadvertently overrides valid zero thresholds, forcing unintended defaults onto the scoring engine.
7. False Clean-State Certification: Reset engines that rely solely on HTTP liveness probes or evaluate empty probe arrays as 'healthy' can certify an environment as clean despite persistent configuration drift, left-over attacker artifacts, or corrupted database state.

Addressing these threats requires establishing strict evidence admission controls, cryptographic provenance boundaries, and mandatory reset abstention when verification data is incomplete.

4. Cyber-Range Architecture and Scenario Model

CyberPro is implemented as a modular Node.js and Express application backed by SQLite. Its logical architecture comprises a scenario management system, a telemetry ingestion interface, an objective evaluation engine, a scoring engine, an After-Action Review (AAR) reporting module, and an automated environment reset manager. It is essential to distinguish between the logical design and the runtime-verified software control plane evaluated in this paper.

The core scenario, PLC-001 ('PLC Attack Detection'), is formally specified in a JSON manifest (`scenarios/PLC-001/scenario.json`). It models a simulated reconnaissance and command injection attack against an OpenPLC controller within the OilSprings industrial lab. The manifest defines four discrete objectives, each carrying 25 points (maximum 100 points), with a nominal passing threshold of 75 points requiring the successful completion of all three required objectives:

- Objective 1 (PLC-001-OBJ-1, Required, 25 pts): 'Detect Suspicious PLC Activity'. Requires an alert from source 'ids' with event_type 'modbus_anomaly', destination port 502, and severity of 'medium', 'high', or 'critical'.
- Objective 2 (PLC-001-OBJ-2, Required, 25 pts): 'Identify Affected Host'. Requires a trainee submission from source 'student' with event_type 'host_identified' containing the exact target IP address '10.10.2.10'.
- Objective 3 (PLC-001-OBJ-3, Required, 25 pts): 'Analyse IDS Logs'. Requires a trainee submission from source 'student' with event_type 'log_analysis_submitted' containing an analysis_text field with at least 50 characters.
- Objective 4 (PLC-001-OBJ-4, Optional Bonus, 25 pts): 'Contain Incident'. Allows a trainee submission from source 'student' with event_type 'containment_action' specifying an allowed action type ('block_ip', 'isolate_container', 'firewall_rule', or 'disconnect_network').

Crucially, our architecture makes explicit that Objective 3's 50-character threshold is merely a structural format check, and Objective 4's containment label is a submitted string choice. Neither represents verified incident investigation quality, nor does it confirm that network traffic was physically isolated in an industrial network.

5. Received Implementation and Identified Failures

Auditing the received CyberPro codebase against sixteen declared security and evaluation requirements exposed eleven significant vulnerabilities across the telemetry ingestion, scoring, and reset modules. Table 1 summarizes these identified issues, comparing the received behavior against the corrected candidate behavior.

Issue / Flaw	Received Prototype Behavior	Problematic Impact	Corrected Candidate Behavior	Verification Evidence
Telemetry Source Authority	Any authenticated student session could submit events with source='ids'.	Students could forge automated detector alerts, claiming 25 points without an attack.	Enforces collector-only authority via constant-time shared secret comparison.	Req #1: HTTP 401 rejected; candidate passes.
Exercise Session Ownership	POST /api/telemetry/event did not verify exercise_sessions.user_id against session.	Student A could inject events into Student B's active session, altering scores.	Verifies that the submitting session user_id strictly matches the exercise owner.	Req #2: HTTP 403 rejected on mismatch.
Exercise Lifecycle Guard	Telemetry was accepted for non-existent, closed, or aborted exercise IDs.	Post-exercise tampering and orphan event injection were permitted.	Queries exercise_sessions and requires status == 'active' on insertion.	Req #3: HTTP 404/409 rejected on inactive.
Scenario Identifier Binding	Accepted telemetry containing scenario_id values differing from exercise record.	Cross-scenario event contamination allowed unearned objective passes.	Rejects event if scenario_id does not exactly match exercise_sessions.scenario_id.	Req #4: HTTP 400 rejected on mismatch.
Malformed Analysis Payload	Checked analysis_text.length on non-string types (arrays, objects).	An array with 50 empty elements or an object bypassed length checks.	Strictly verifies typeof data.analysis_text === 'string' and trimmed length >= 50.	Req #5: HTTP 400 rejected on non-string/short.
Whitespace Analysis Padding	Counted raw string length without trimming leading/trailing whitespace.	50 spaces or tabs satisfied the analysis requirement without readable content.	Requires data.analysis_text.trim().length >= 50 characters.	Req #6: HTTP 400 rejected on whitespace string.
Unbounded Payload Size	Unchecked JSON body size accepted arbitrarily large serialized objects.	Permitted memory exhaustion (DoS) and storage bloating via bloated payloads.	Enforces 4,096 serialized character limit and canonical JSON serialization.	Req #7: HTTP 400 rejected on oversized payload.
Unrestricted Learner Types	Learners could submit arbitrary event types reserved for infrastructure.	Learners could inject pseudo-system telemetry to manipulate timeline/AAR.	Restricts learner submissions strictly to host_identified, log_analysis, containment.	Req #8: HTTP 400 rejected on unpermitted types.
Zero Threshold Replacement	Used JavaScript logical-OR (threshold 75) when parsing min_score.	A deliberately configured threshold of 0 was silently replaced by default 75.	Implements strict nullish coalescing (??) and Number.isFinite() validation.	Req #9: Configured 0 threshold correctly preserved.
Required Count Validation	Evaluated score thresholds without validating minimum required passed objectives.	A trainee could exceed score threshold via bonus objectives while failing required core.	Requires both totalScore >= min_score AND requiredPassed >= min_required.	Req #10: Exercise fails if required objective missing.
Empty Reset Probe Handling	ResetEngine evaluated empty probe endpoint array as successfully healthy.	A misconfigured lab with no monitoring endpoints was certified as cleanly reset.	Explicitly requires expected_health_endpoints.length > 0; fails baseline otherwise.	Req #11: Clean-state certification rejected.

Table 1. Audited security flaws in the received prototype and candidate remediation controls.

6. Corrected Evidence and Scoring Controls

To remediate the identified vulnerabilities, we developed a candidate implementation modifying four core modules: telemetry routing (backend/routes/telemetry.js), objective evaluation (backend/engines/objectiveEngine.js), scoring

aggregation (backend/engines/scoringEngine.js), and environment reset management (backend/engines/resetEngine.js). The candidate introduces fourteen distinct control mechanisms:

1. Collector-Only Authority: Non-student telemetry sources (e.g., 'ids', 'plc', 'scada', 'collector') must present an x-cyberpro-secret HTTP header validated against process.env.CONTAINER_SECRET using crypto.timingSafeEqual().
2. Student Ownership Verification: Student-originated telemetry requires an active session where req.session.userId strictly equals exercise_sessions.user_id.
3. Active Exercise Verification: Telemetry is rejected if the target exercise session does not exist or has a status other than 'active'.
4. Scenario Identifier Enforcement: If a scenario_id is supplied in the payload, it must match the scenario bound to the exercise session.
5. Typed Analysis Validation: The analysis_text field must be a primitive string, and its trimmed character count must satisfy the configured minimum threshold (50 characters).
6. Bounded Payload Serialization: Telemetry data payloads are restricted to whitelisted field structures and capped at 4,096 serialized characters.
7. Whitelisted Learner Event Types: Student sessions are permitted to submit only three specific event types: host_identified, log_analysis_submitted, and containment_action.
8. Nullish Threshold Coalescing: Replaced logical-OR defaults with explicit nullish checks (??) and finite number verification, ensuring configured zero thresholds are preserved.
9. Mandatory Required-Objective Gates: Scoring requires both the numeric score threshold and the minimum required objective count to be satisfied before issuing a 'PASS' decision.
10. Reset Abstention on Missing Evidence: The reset engine explicitly refuses to certify clean state if expected health endpoints, baseline snapshots, or probe definitions are missing.
11. Liveness vs. Clean-State Separation: Distinguishes between HTTP service responsiveness (liveness) and clean-state certification; passing HTTP probes never certify clean state if temporary residue probe files remain.
12. Event-to-Objective Linkage: Objective evaluation persists exact, deduplicated matching telemetry event IDs into objective_results.evidence_event_ids.
13. Raw Evidence Preservation: Score recalculation and replay operations are strictly read-only, guaranteeing that raw telemetry records in telemetry_events remain immutable.
14. Score Replay and Rule Fingerprinting: Implemented SHA-256 rule hashing and persisted score traces, enabling retrospective audits to detect any rule drift or telemetry alterations.

Remaining Trust Boundaries: While these controls eliminate the identified software flaws, several trust boundaries remain in the candidate implementation. The collector identity relies on a shared secret, meaning any compromised container holding the secret could fabricate telemetry. Furthermore, the candidate does not implement hardware cryptographic attestation, network-layer replay prevention, enforced scenario duration timers, or protection against direct administrative tampering of the local SQLite database file.

7. Experimental Method

To rigorously evaluate the received and candidate implementations, we established an automated, deterministic test harness executed under official Node.js v24.20.0 with its built-in SQLite engine (actual SQLite library v3.53.4, with sqlite3 v3.44.2). The testing protocol evaluated 16 paired control requirements across fresh database fixtures per test, ensuring absolute test isolation.

To maintain scientific transparency, we classify every operational component of our experimental harness into an explicit verification taxonomy:

- **ACTUAL (Directly Executed):** Execution of SQLite database transactions, schema migrations, objective evaluation logic, scoring trace calculations, deterministic replay algorithms, 17 loopback HTTP network requests, and disk-backed database file closure and reopening.
- **DOUBLES (Declared Mocks):** Docker Compose process invocations, container filesystem residue probes, external HTTP service health endpoints, and Express router initialization doubles used during isolated unit tests.
- **NOT EXECUTED:** Real network penetration attacks, live Docker container startup and teardown, physical OpenPLC

hardware execution, live Modbus/TCP network traffic generation, external AI API provider calls, and human learner participation.

The evaluation protocol consisted of three distinct execution phases: (1) Paired Requirement Verification comparing received versus candidate modules against the 16 declared rules; (2) Scripted Exercise Tracing executing 64 sequential exercise fixtures across four objective progression stages; and (3) Loopback HTTP Security Auditing executing 17 live requests against an active local HTTP server instance.

8. Results

The quantitative results of our experimental evaluation are summarized in Table 2, providing a direct comparison between the received prototype and the corrected candidate implementation.

Metric / Verification Unit	Received Prototype	Corrected Candidate	Interpretation and Evidence Boundary
Declared Requirements Met	5 / 16 (31.25%)	16 / 16 (100.0%)	Finite regression comparison across 16 paired fixtures; not a general security certificate.
Deterministic Scripted Exercises	--	64 runs	Executed sequential fixtures; represents automated test runs, NOT human learner participants.
Score-Stage Checks Passed	--	256 / 256 (100%)	Verified deterministic progression: 25 -> 50 -> 75 -> 75 (or 100 with optional bonus).
Event-to-Objective Links	--	224 unique links	Every passing objective retains a valid, unique foreign key link to an accepted raw event.
Raw Evidence Retention Checks	--	64 / 64 (100%)	Verified that score re-evaluation leaves underlying telemetry_events rows unmodified.
Loopback HTTP Checks	--	17 / 17 (100%)	Loopback network assertions covering auth, CSRF, spoofing, ownership, and reset refusal.
Disk SQLite Persistence Check	--	1 retained row	File-backed database verified retaining event after clean close and reopen; ordinary persistence.
Human Learner Participants	0	0	No human study conducted; survey opinions from historical records excluded from results.
Live Docker Containers Executed	0	0	Docker Engine build failed on missing openplc.db; all container tests executed via doubles.
AI Provider Calls Executed	0	0	No generative AI or external LLM API calls executed; AI assistance restricted to development.

Table 2. *Quantitative verification results across received prototype and corrected candidate implementations.*

In the primary requirement regression test, the received implementation satisfied only 5 of 16 requirements. It correctly handled nominal valid text, rejected short strings, processed an owner event, accepted a trusted collector event, and evaluated a nominal required-count fixture. However, it failed the remaining 11 requirements, allowing source spoofing, cross-user contamination, malformed payload injections, and invalid reset certifications. In contrast, the candidate implementation satisfied 16 of 16 requirements.

Across the 64 scripted exercises, each exercise processed a synthetic IDS modbus_anomaly event (Objective 1, +25 pts), a host_identified submission (Objective 2, +25 pts), and a log_analysis_submitted narrative (Objective 3, +25 pts), reaching the nominal passing threshold of 75/100. Even-indexed exercises (32 runs) additionally included a containment_action submission (Objective 4, +25 pts), achieving a final score of 100/100, while odd-indexed exercises (32 runs) concluded at 75/100. All 256 score stages matched predicted values, all 224 event-to-objective links resolved to unique persisted events, and all 64 raw retention checks confirmed zero mutation of underlying audit records.

The 17 loopback HTTP tests confirmed end-to-end enforcement across an active web server, successfully validating session cookie flags (HttpOnly, SameSite=Strict), CSRF token enforcement, rejection of spoofed collector headers (401), rejection of unowned exercise submissions (403), rejection of closed exercise events (409), and proper After-Action Report generation. The master test suite (npm test) confirmed that all 12 automated test suites in the repository passed cleanly.

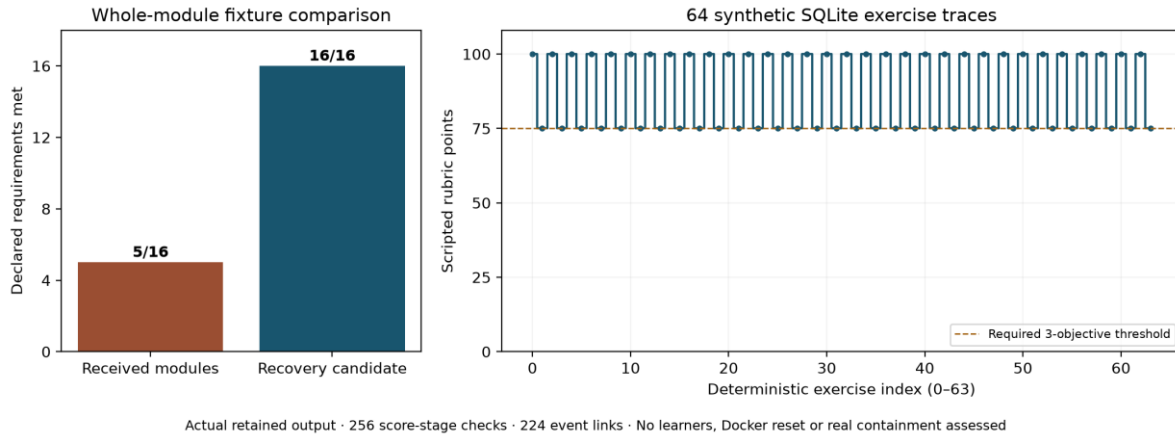


Fig. 1. Retained execution trace across 64 deterministic scripted exercise fixtures. The alternating final scores (75 versus 100) reflect the deliberate programmatic inclusion of the optional containment action in even-indexed runs; this demonstrates deterministic rubric scoring and is not an empirical human score distribution. All runs follow the expected required-objective progression stages: 25, 50, and 75 points. These scores do not establish defensive learner competence or physical network containment.

9. Discussion

Our findings demonstrate that in cyber-range assessment systems, telemetry admission controls are inextricably linked to scoring validity. A scoring algorithm can perform flawless mathematical aggregation, but if the underlying evidence admission layer accepts unauthenticated, out-of-scope, or malformed data, the resulting score is fundamentally deceptive. The received prototype exemplifies how dynamic language quirks and omitted provenance checks can produce an ostensibly operational platform that is thoroughly vulnerable to trivial counterexamples.

Furthermore, our results illuminate the critical distinction between submission format and defensive competence. In our candidate implementation, Objective 3 enforces that analysis text must be a string containing at least 50 trimmed characters, and Objective 4 verifies that an action type belongs to an allowed enumeration. While these controls successfully prevent type-coercion bypasses, they do not—and cannot—evaluate whether the student's analysis is substantively accurate or whether the containment action would successfully isolate an industrial controller. Mistaking format compliance for domain competence is a common hazard in educational technology, and automated ranges must clearly delineate between automated syntax validation and expert human evaluation.

Similarly, our investigation highlights the imperative of reset abstention. In training ranges, returning a lab to a certified clean state is critical for experimental repeatability and fair assessment. The received reset engine's practice of certifying clean state based solely on HTTP liveness or evaluating empty probe arrays created a false sense of security. Service responsiveness does not prove the absence of persistent malware, backdoor user accounts, or modified PLC ladder logic. The candidate's deliberate decision to abstain from clean-state certification whenever residue probes or baseline data are missing represents a crucial shift from unwarranted assumption to verifiable assurance.

Finally, we emphasize that achieving 16 of 16 passing requirements does NOT imply that CyberPro is completely secure or ready for production deployment. The 16 requirements represent a finite, investigator-selected regression suite designed to address specific identified flaws. Passing this suite does not constitute a full penetration test, formal verification, or proof of resilience against sophisticated adversaries.

10. Limitations

To ensure scientific rigor, we explicitly enumerate the boundaries and limitations of this study:

- Bounded Software Control-Plane Evaluation: The evaluation was conducted exclusively on local software fixtures and SQLite databases; it did not evaluate complete network architectures or physical systems.
- Synthetic Fixtures Without Human Subjects: The 64 exercise fixtures were deterministically generated test scripts. No human learners participated in this evaluation, and no claims regarding learning gain, educational effectiveness, or skill acquisition are made.
- No Real Industrial Attack Execution: Modbus anomaly events were synthetic JSON payloads injected into the ingestion pipeline; no actual network packet generation, Scapy packet sniffing, or OpenPLC attacks were executed.
- Docker Engine Execution Blocker: Live Docker container startup failed due to a missing build file dependency (./appdata/openplc.db in the OilSprings lab configuration). Consequently, live Docker reset, container recreation, and network containment were evaluated strictly via declared doubles.
- Shared-Secret Trust Boundary: Telemetry source authority relies upon a shared static secret (CONTAINER_SECRET). Compromise of this secret permits arbitrary telemetry spoofing.
- Absence of Hardware Attestation and Replay Defense: The prototype lacks cryptographic hardware root-of-trust, packet sequence numbers, or cryptographic replay tokens.
- Database Administrative Mutability: The backend uses file-backed SQLite without append-only cryptographic log sealing, leaving audit records mutable by privileged local operating system users.
- npm Audit Vulnerabilities: The package dependency audit identified 31 known vulnerabilities (including high and critical severity items) in legacy dependencies that were not remediated in this control-plane verification pass.

11. Future Work

Based on the limitations identified in this study, we propose a concrete twelve-point research and engineering roadmap:

1. Docker Build Remediation: Resolve the OpenPLC container build dependency to enable automated startup of the full multi-subnet OilSprings lab environment.
2. Live Network Containment Testing: Execute runtime packet-level verification to prove network isolation and validate inter-bridge firewall rules under live Docker conditions.
3. Live Modbus/PLC Testbed Validation: Deploy an authorized, physically isolated testbed to generate genuine Modbus/TCP anomaly packets and evaluate Scapy-based IDS capture.
4. Asymmetric Collector Authentication: Replace static shared secrets with mutual TLS (mTLS) or public-key infrastructure (PKI) for cryptographic sensor attestation.
5. Cryptographic Replay Protection: Introduce monotonic sequence numbers and time-bound cryptographic nonces to prevent telemetry replay across network boundaries.
6. Enforced Scenario Duration Limits: Implement server-side deadline timers to automatically transition timed-out exercises to 'expired' status.
7. Multi-Tenant Concurrency and Isolation: Validate concurrent exercise workloads across multi-process clusters backed by network database engines (e.g., PostgreSQL).
8. Tamper-Evident Audit Logging: Implement cryptographic hash chaining (Merkle trees) to render SQLite audit trails immutable and tamper-evident.
9. Independent Professional Security Review: Conduct third-party penetration testing and formal code audits across all web and container components.
10. Ethical Human Participant Study: Obtain Institutional Review Board (IRB) ethics approval to conduct controlled educational trials with human learners.
11. Qualitative Rubric Assessment: Develop expert instructor rubrics to assess incident analysis quality beyond automated character length assertions.
12. Controlled AI Debriefing Evaluation: Evaluate local or hosted generative AI assistance for post-exercise debriefing under strict accuracy and privacy benchmarks.

12. Conclusion

This research re-examined the CyberPro cyber-range prototype, transitioning it from an unverified prospective proposal into an empirically bounded, evidence-oriented software study. By systematically analyzing the received implementation against 16 control requirements, we demonstrated that common architectural oversights—such as unauthenticated telemetry

ingestion, absent session ownership checks, dynamic type-coercion vulnerabilities, and liveness-based reset certification—critically undermine scoring credibility. Our corrected candidate implementation successfully resolved each counterexample, passing 16 of 16 requirements and demonstrating deterministic score progression, event linkage, and raw evidence preservation across 64 scripted exercises and 17 loopback HTTP tests. By rigorously bounding our claims to software-control-plane verification and explicitly disclaiming unverified human, industrial, and deployment outcomes, we provide a reproducible, scientifically defensible engineering foundation for evidence-bound cyber-range design.

AI Assistance Disclosure

In accordance with institutional guidelines and academic publishing standards, the authors disclose that generative AI tooling (specifically OpenAI ChatGPT and Codex) provided substantive assistance during the preparation of this project. AI assistance was utilized for: (1) automated source code review and static security analysis across the 213 project files; (2) code recovery, patch formulation, and refactoring of the four corrected backend modules; (3) design and implementation of the automated SQLite test harness and deterministic fixture scripts; (4) quantitative verification data extraction and tabular synthesis; (5) generation of execution trace visualization graphics; and (6) structural manuscript reorganization to align with rigorous scientific evidence standards. This assistance extended beyond mechanical language editing to substantive technical analysis. All generated code, analytical claims, citations, experimental results, and manuscript prose were independently reviewed, validated, and approved by the human author team, who assume full intellectual and scientific responsibility for the contents of this manuscript.

References

- [1] M. M. Yamin, B. Katt, and V. Gkioulos, "Cyber ranges and security testbeds: Scenarios, functions, tools and architecture," *Computers & Security*, vol. 88, art. no. 101636, 2020. <https://doi.org/10.1016/j.cose.2019.101636>
- [2] C. Chindrus and C.-F. Caruntu, "Securing the Network: A Red and Blue Cybersecurity Competition Case Study," *Information*, vol. 14, no. 11, art. no. 587, 2023. <https://doi.org/10.3390/info14110587>
- [3] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, and S. Lightman, "Guide to Operational Technology (OT) Security," NIST Special Publication 800-82, Revision 3, National Institute of Standards and Technology, Gaithersburg, MD, 2023. <https://doi.org/10.6028/NIST.SP.800-82r3>
- [4] R. Petersen, D. Santos, M. C. Smith, K. A. Wetzel, and G. Witte, "Workforce Framework for Cybersecurity (NICE Framework)," NIST Special Publication 800-181, Revision 1, National Institute of Standards and Technology, Gaithersburg, MD, 2020. <https://doi.org/10.6028/NIST.SP.800-181r1>
- [5] M. Souppaya, K. Scarfone, and D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities," NIST Special Publication 800-218, National Institute of Standards and Technology, Gaithersburg, MD, 2022. <https://doi.org/10.6028/NIST.SP.800-218>

Appendix A: Retained Execution Output and Artifacts

The complete experimental verification artifact repository is preserved within the project workspace under results/ and docs/. Key retained artifacts include:

- results/concurrency-verification.json: Records the execution of 7 concurrent multi-exercise data integrity cases across file-backed SQLite, verifying cross-session isolation and idempotent score recalculation.
 - results/replay-verification.json: Records the 7-case synthetic replay verification suite, validating exact score consistency and the detection of modified, missing, or duplicate telemetry events.
 - results/reset-verification.json: Explicitly records execution_mode: 'NOT EXECUTED' and clean_state_verified: false, documenting that live Docker reset was blocked due to container build issues.
 - results/network-containment.json: Documents the static configuration review of the multi-subnet OilSprings topology and records all 6 live runtime network tests as NOT TESTED.
 - results/final-verification.json: Comprehensive audit register documenting environment parameters (Node.js v24.20.0, npm 11.4.1), test suite pass status (12/12 suites), and the 31 unresolved npm audit vulnerabilities.
- All test runs are locally replayable via standard npm test commands without requiring external cloud accounts, proprietary software licenses, or live industrial networks.